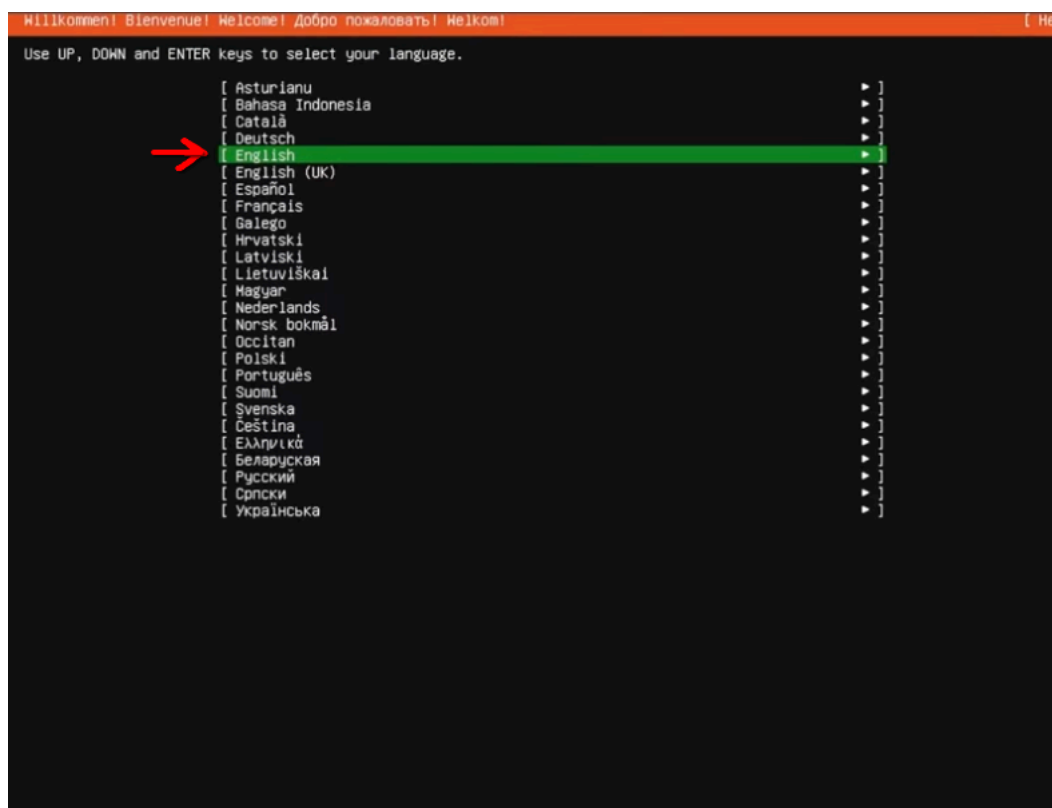
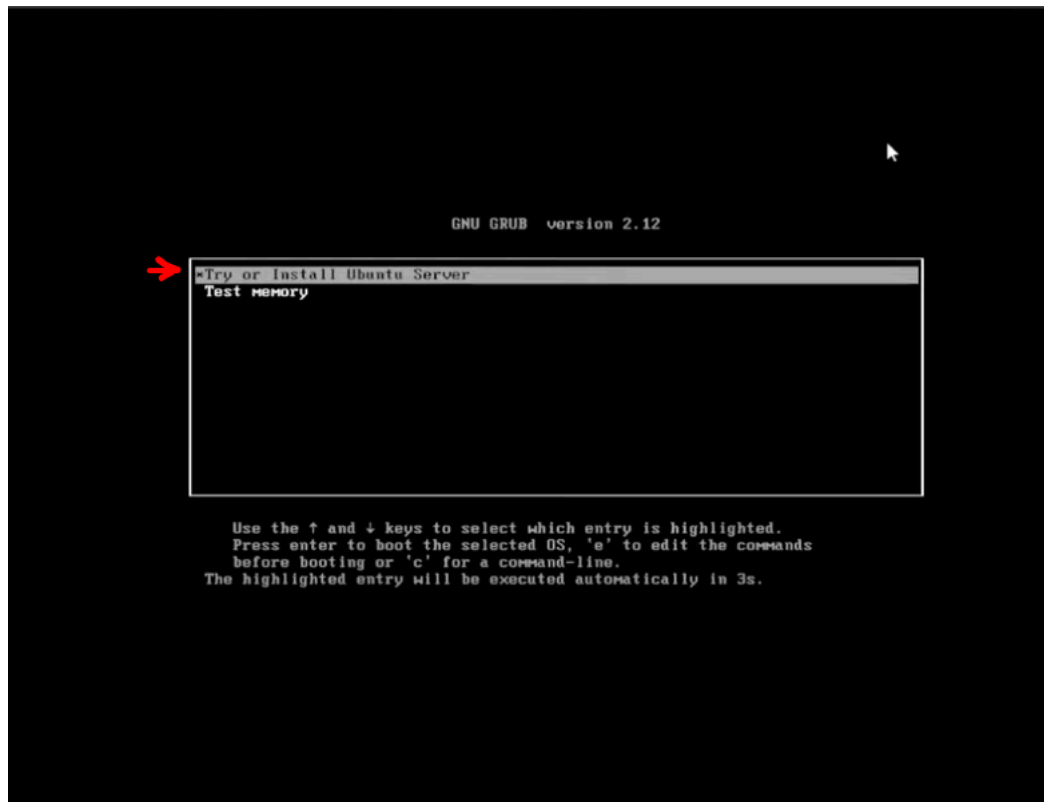
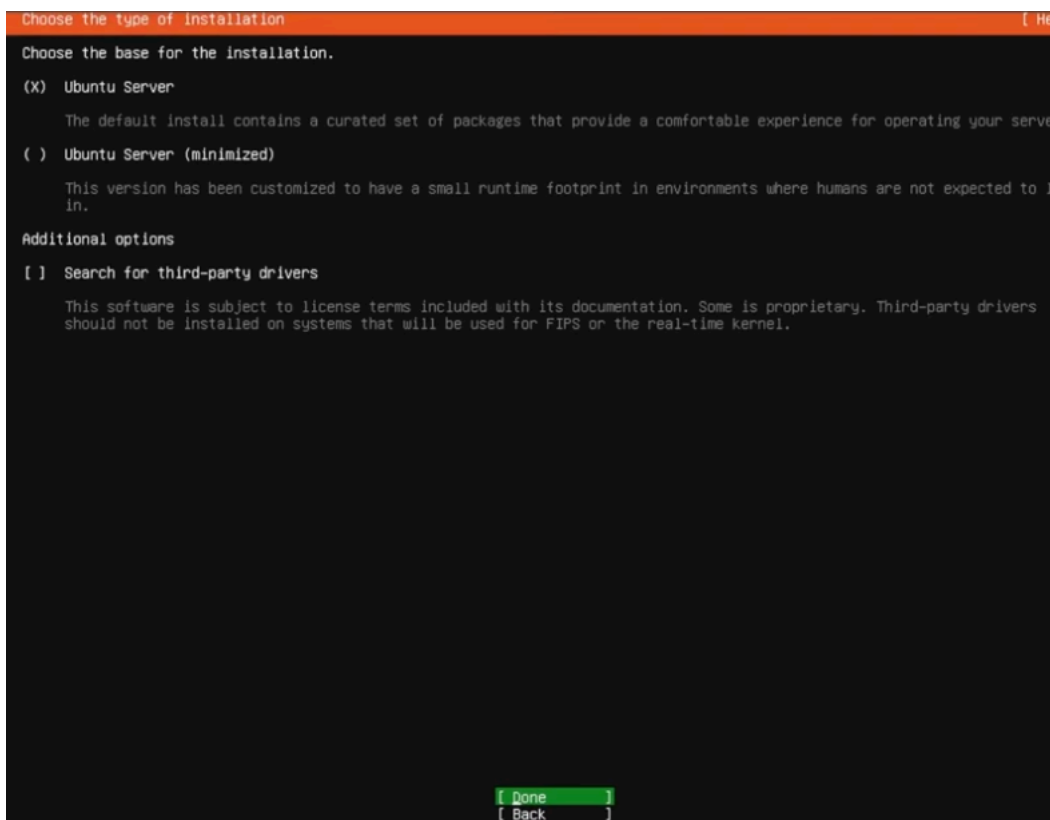
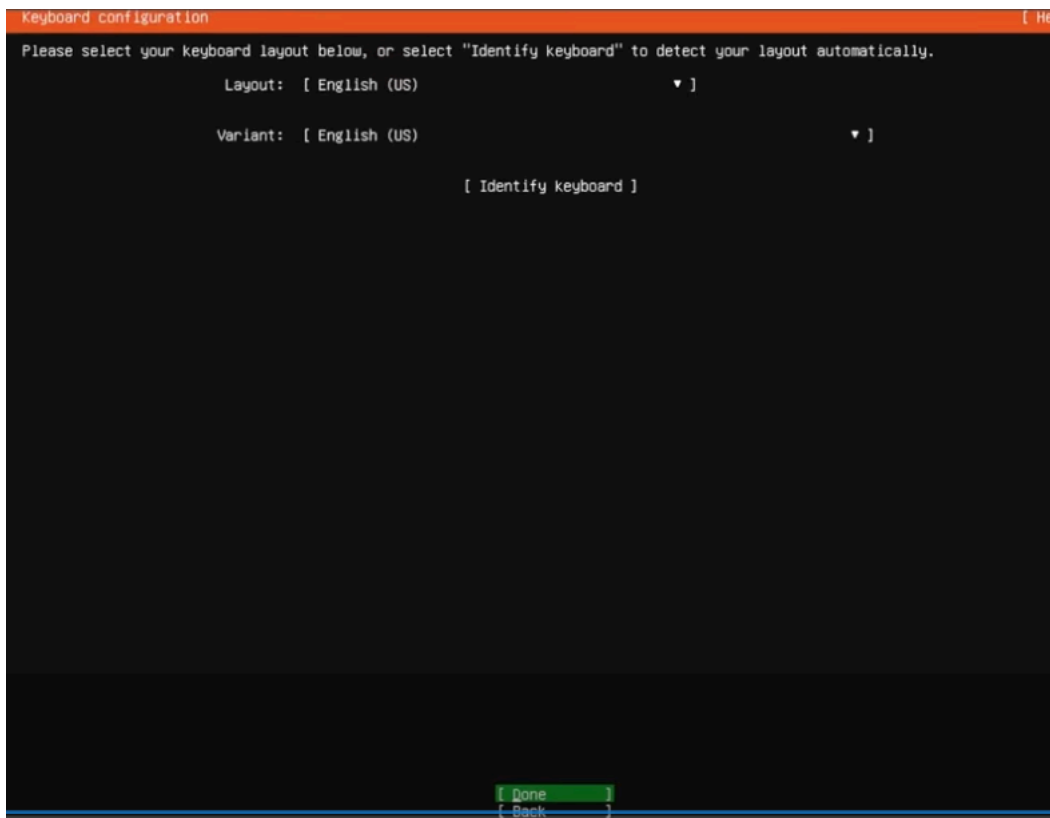
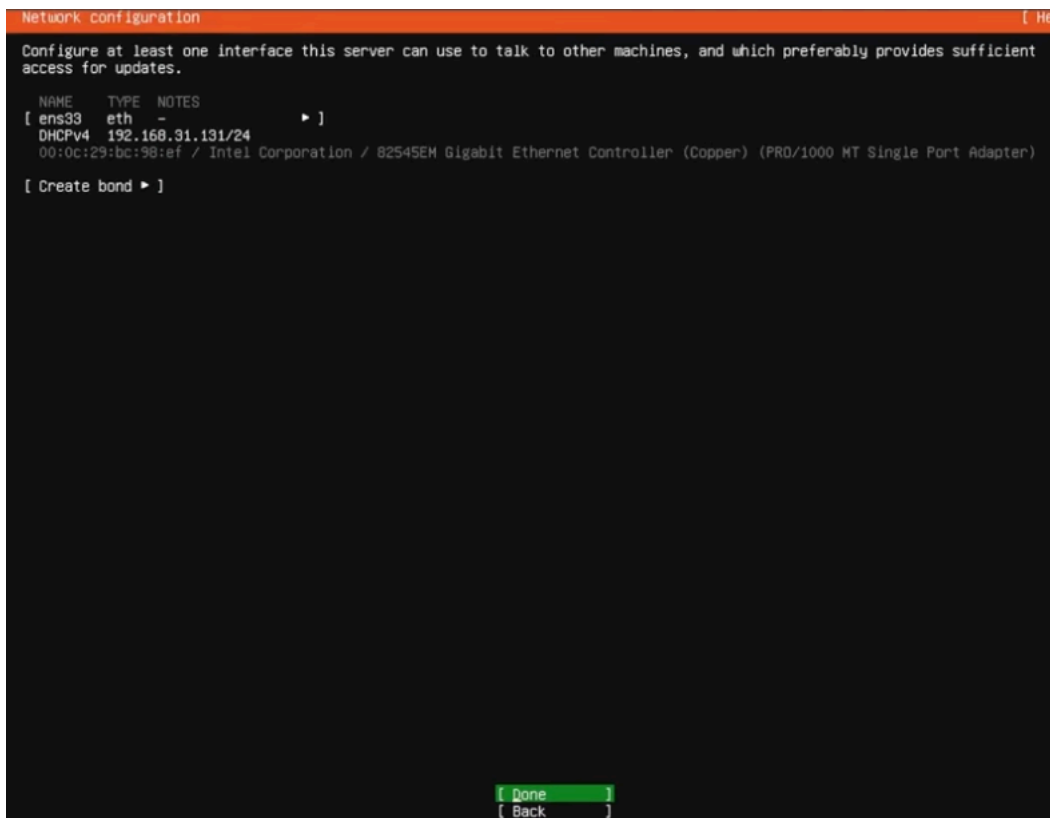


1.ติดตั้ง Ubuntu Server

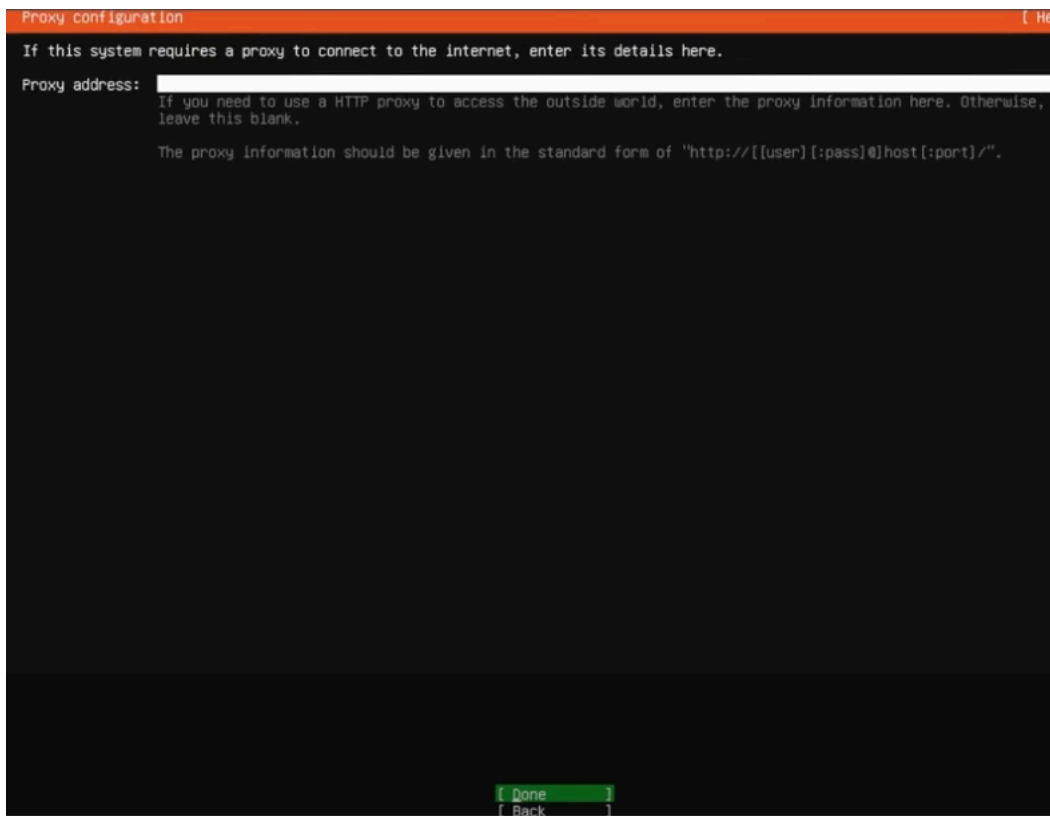


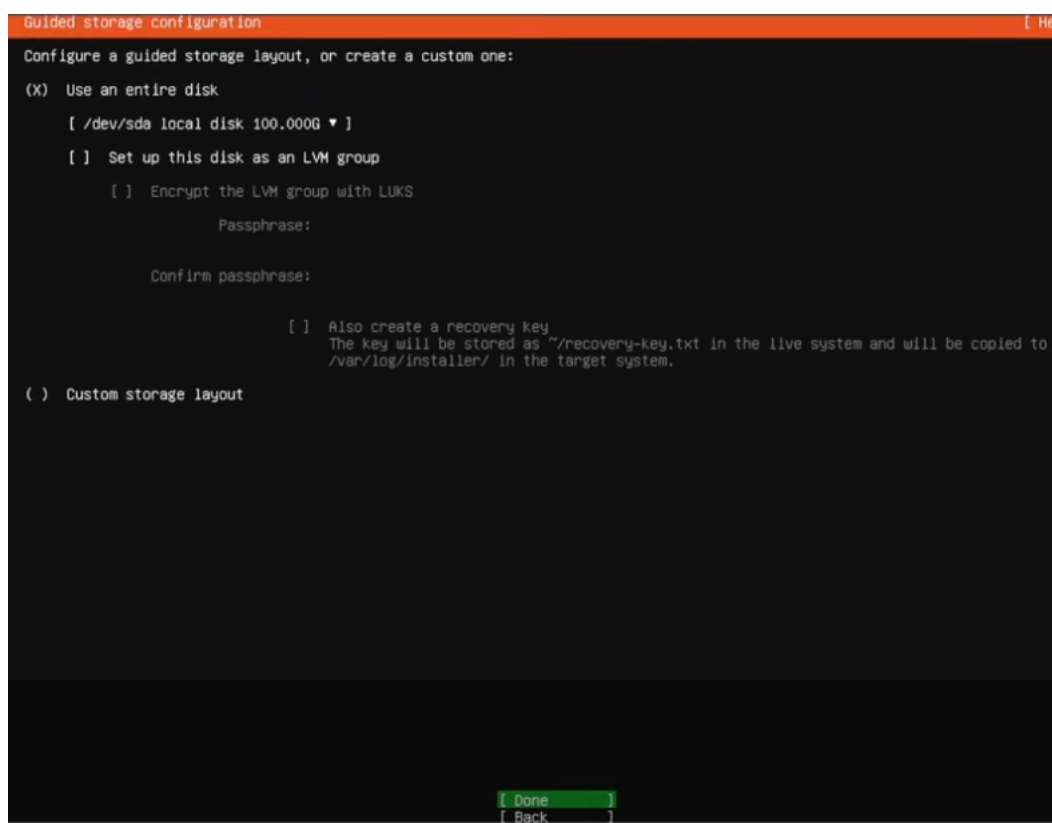
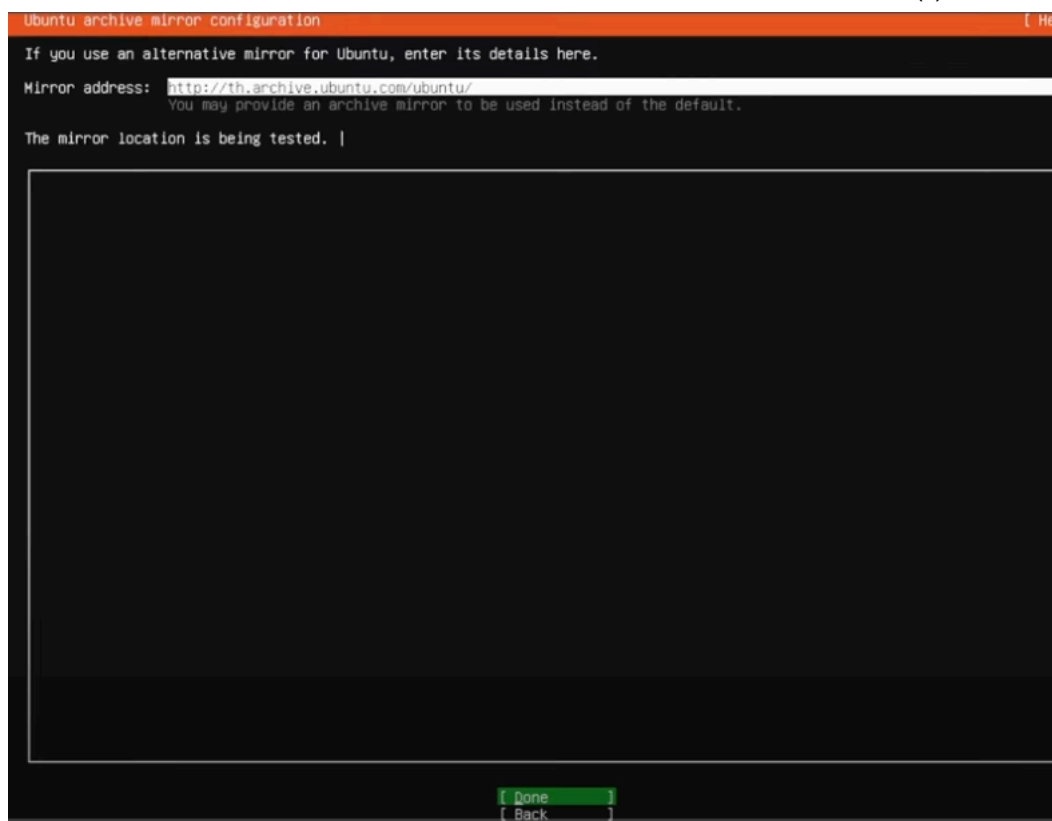


เลือก Ubuntu server

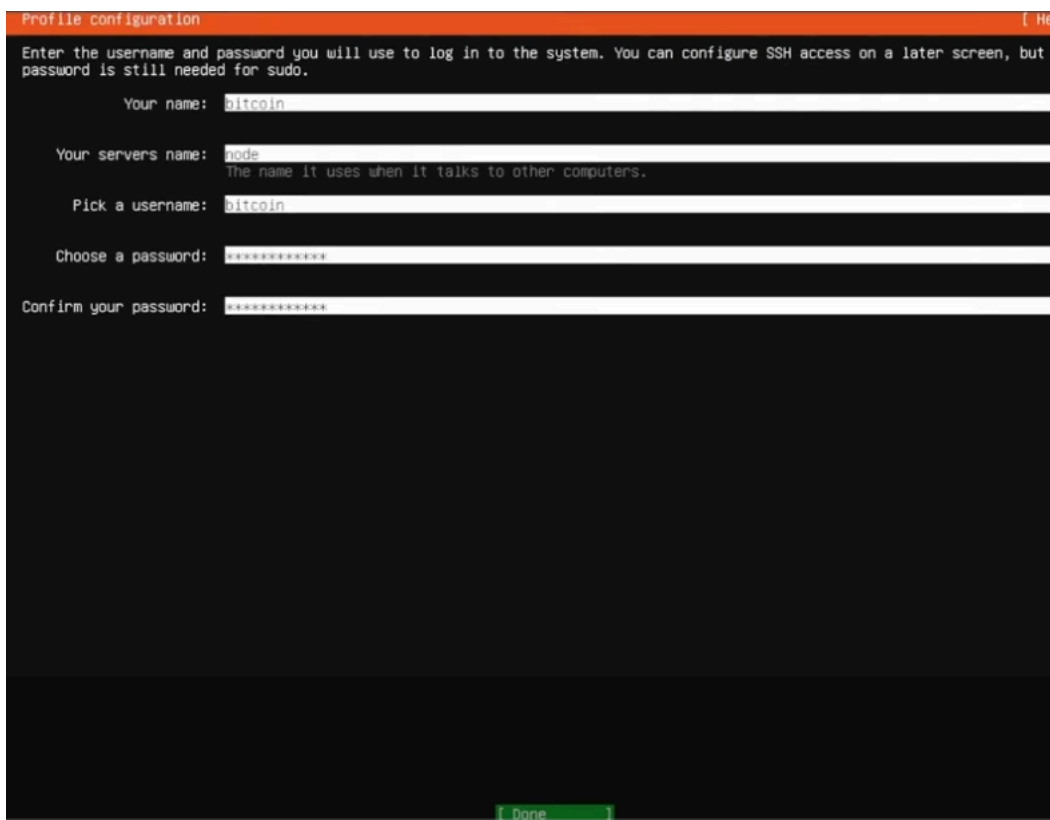
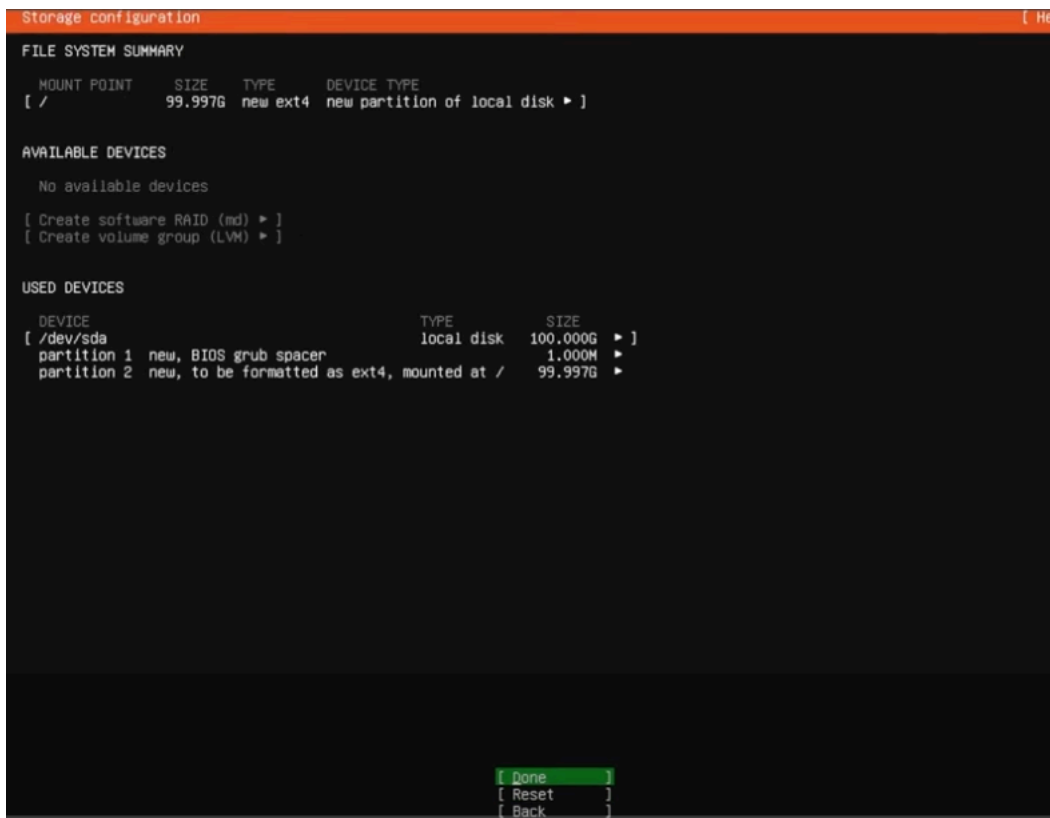


(จด IP เอาไว้ก็ได้)





เลือกตัวเลือกตามที่เห็นบนหน้าจอ ให้มีแค่กากบาทเดียว



!!!ขั้นตอนสำคัญ!!!

ให้จด username และ password ที่ตั้งเอาไว้ด้วย

```
SSH configuration [ He

You can choose to install the OpenSSH server package to enable secure remote access to your server.

[X] Install OpenSSH server

[X] Allow password authentication over SSH

[ Import SSH key ► ]

AUTHORIZED KEYS

No authorized key

[ Done ]
[ Back ]
```

```
Featured server snaps [ He

These are popular snaps in server environments. Select or deselect with SPACE, press ENTER to see more details of the pack
publisher and versions available.

[ ] microk8s          canonical✓    Kubernetes for workstations and appliances
[ ] nextcloud         nextcloud✓   Nextcloud Server - A safe home for all your data
[ ] wekan             xet7        Open-Source kanban
[ ] kata-containers  katacontainers✓ Build lightweight VMs that seamlessly plug into the containers ecosystem
[ ] docker           canonical✓   Docker container runtime
[ ] canonical-livepatch canonical✓    Canonical Livepatch Client
[ ] rocketchat-server rocketchat✓   Rocket.Chat server
[ ] mosquitto        mosquitto✓   Eclipse Mosquitto MQTT broker
[ ] etcd             canonical✓   Resilient key-value store by CoreOS
[ ] powershell       canonical✓   PowerShell for every system!
[ ] sabnzbd          safihre      SABnzbd
[ ] wormhole         snapcrafters get things from one computer to another, safely
[ ] aws-cli          aws✓         Universal Command Line Interface for Amazon Web Services
[ ] google-cloud-sdk google-cloud-sdk✓ Google Cloud SDK
[ ] slcli            softlayer    Python based SoftLayer API Tool.
[ ] doctl            digitalocean✓ The official DigitalOcean command line interface
[ ] postgresql10     cmd✓         PostgreSQL is a powerful, open source object-relational database system.
[ ] heroku           heroku✓      CLI client for Heroku
[ ] keepalived       keepalived-project✓ High availability VRRP/BFD and load-balancing for Linux
[ ] prometheus       canonical✓   The Prometheus monitoring system and time series database
[ ] lxd              canonical✓   LXD - container and VM manager

[ Done ]
[ Back ]
```

```

Installation complete!

writing install sources to disk
running 'curtin extract'
  curtin command extract
    acquiring and extracting image from cp:///tmp/tmpqz1lc3j/mount
configuring keyboard
curtin command in-target
executing curtin install curthooks step
curtin command install
  configuring installed system
  running 'curtin curthooks'
  curtin command curthooks
    configuring apt configuring apt
    installing missing packages
    installing packages on target system: ['grub-pc']
    configuring iscsi service
    configuring raid (mdadm) service
    configuring NVMe over TCP
    installing kernel
    setting up swap
    apply networking config
    writing etc/fstab
    configuring multipath
    updating packages on target system
    configuring pollinate user-agent on target
    updating initramfs configuration
    configuring target system bootloader
    installing grub to target devices
    copying metadata from /cdrom
final system configuration
calculating extra packages to install
installing openssh-server
retrieving openssh-server
curtin command system-install
unpacking openssh-server
curtin command system-install
configuring cloud-init
downloading and installing security updates
curtin command in-target
restoring apt configuration
curtin command in-target
subiquity/Late/run:

[ View full log ]
[ Reboot Now ]

```

แล้วก็รอ

2.ต่อโหนดเข้ากับ Wifi

ตอนติดตั้ง Ubuntu Server เราได้ตั้งค่าให้เชื่อมต่อกับ Wi-Fi ของ CDCstaff ไว้เท่านั้น ดังนั้นเมื่อกลับบ้าน เครื่องจะไม่สามารถเชื่อมต่อ Wi-Fi ที่บ้านได้โดยอัตโนมัติ จึงจำเป็นต้องเพิ่มการตั้งค่า Wi-Fi ของบ้านเข้าไปด้วย

2.1. ตั้งค่า WIFI Setting ภายใน Node ของคุณ

```
sudo nano /etc/netplan/50-cloud-init.yaml
```

หลังจากกรอกคำสั่งแล้ว ที่หน้าจอของนักเรียนควรที่จะแสดงข้อความดังต่อไปนี้

```

network:
  version: 2
  wifis:
    wlan0:
      optional: true
      dhcp4: true
      access-points:
        "CDCstaff":
          hidden: true
          auth:
            key-management: "psk"
            password: "6927fa0887ce107f274e36e5640a93e9f33e17d247212bff991ddb459c94384a"

```

หลังจากนั้นทำการพิมพ์แก้ไข

```

network:
  version: 2
  wifis:
    wlan0:
      optional: true
      dhcp4: true
      access-points:
        "CDCstaff":
          hidden: true
          auth:
            key-management: "psk"
            password: "6927fa0887ce107f274e36e5640a93e9f33e17d247212bff991ddb459c94384a"
        "<name-of-your-wifi-home>":
          auth:
            key-management: "psk"
            password: "<name-of-your-wifi-home>"
        "<name-of-your-hotspot>":
          auth:

```

```
key-management: "psk"
password: "<your-hotspot-password>"
```

โดยแทนที่ <name-of-your-wifi-home> และ <name-of-your-hotspot> และ ด้วยชื่อไวไฟที่บ้านคุณ

และแทนที่ <your-wifi-home-password> และ <your-hotspot-password> ด้วยรหัสผ่านเข้าไวไฟที่บ้านคุณ

หลังจากนั้นก็กด Ctl+S และ Ctl+ X เพื่อบันทึก

ถัดมาก็ทำการยืนยันการตั้งค่าดังกล่าวด้วยคำสั่ง

```
sudo netplan apply
```

หลังจากนั้น เชื่อมต่อ Node ของคุณเข้ากับ WIFI ที่ต้องการ โดย

1. ดูว่ามีไวไฟอะไรที่เครื่องเจอบ้าง

```
sudo iwlist wlan0 scan | grep ESSID
```

หลังจากที่กรอกคำสั่ง ควรที่จะมีชื่อ WIFI ของท่านขึ้นตามท้ายอักษร ESSID

```
ESSID: "<name-your-wifi-home>"
ESSID: "<name-your-hotspot>"
```

หลังจากนั้น ก็ทำการเชื่อมต่อ

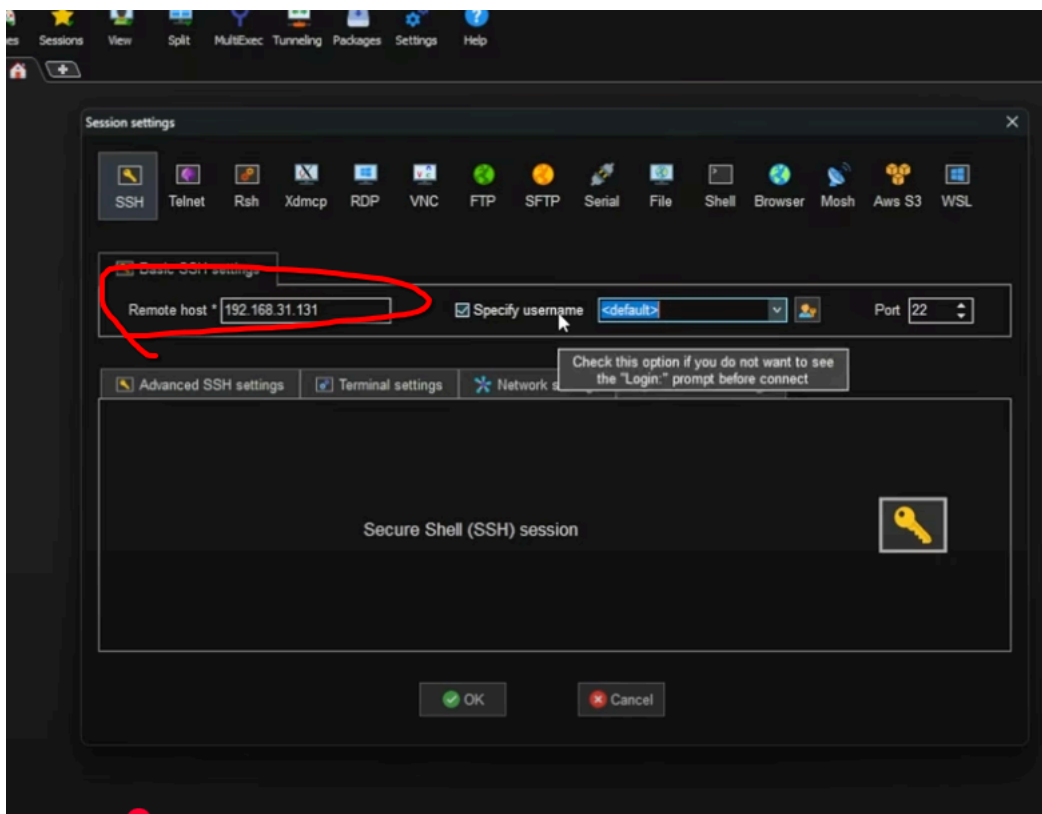
```
nmcli dev wifi connect "<name-your-hotspot>" password "<your-hotspot-password>"
```

โดยแทนที่ <name-your-hotspot> ด้วยชื่อไวไฟของคุณที่แสดงหลัง ESSID และ <your-hotspot-password> ด้วยรหัสผ่านของไวไฟนั้น

3.เข้าใช้โหนดจากคอมพิวเตอร์ส่วนตัวของคุณ

เราจะใช้งาน Node ของเราด้วยคอมพิวเตอร์ส่วนตัวเป็นหลัก

3.1. เปิดโปรแกรม mobaXterm และกรอก ip เข้าไปในช่องตามภาพ

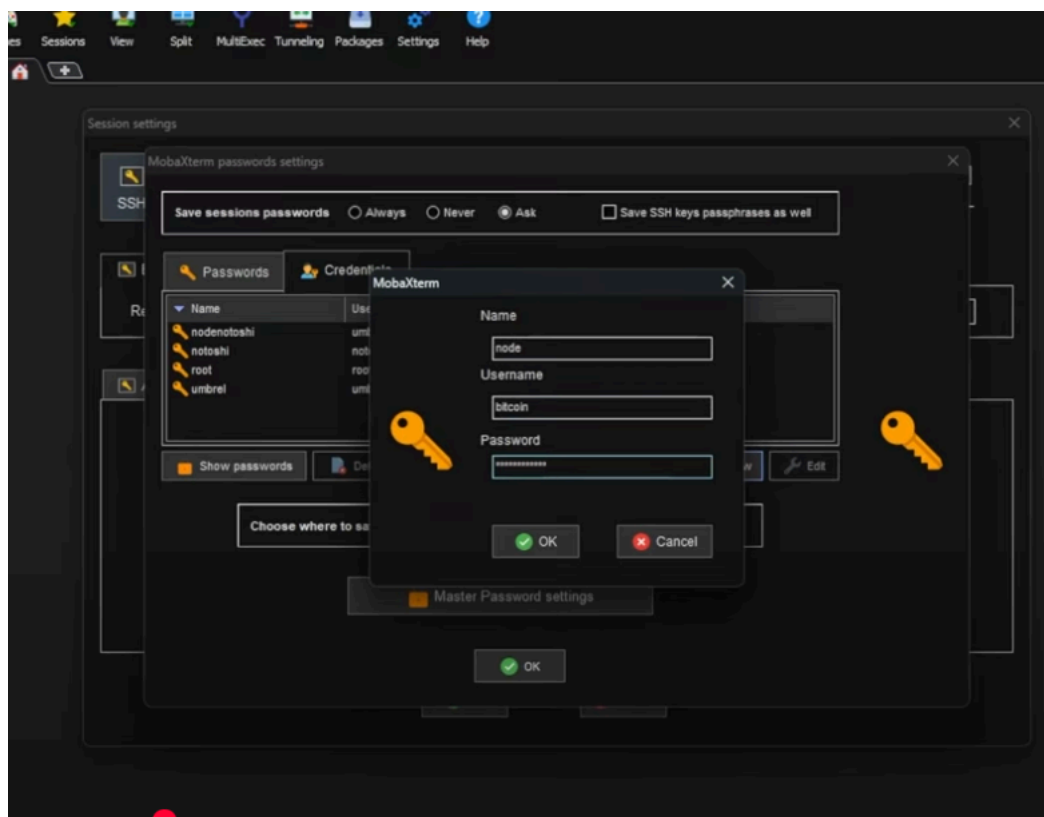
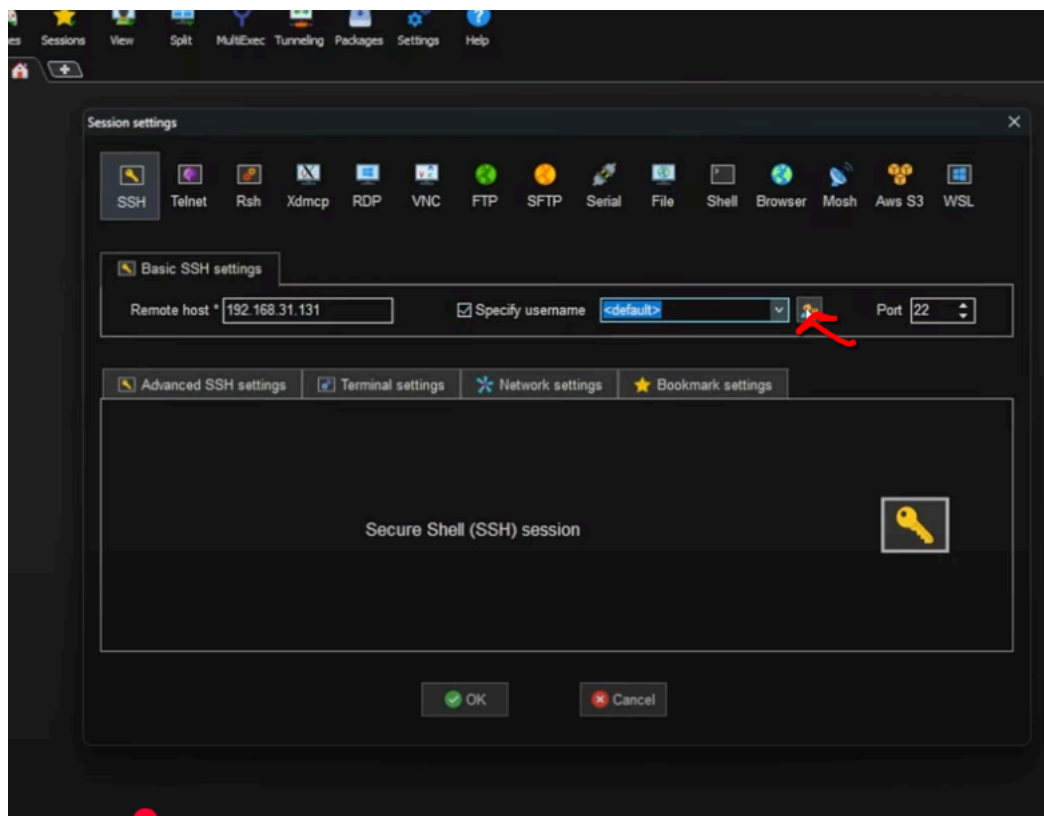


โดยดู IP ที่ต้องใช้ได้จากคำสั่งนี้ใน Terminal

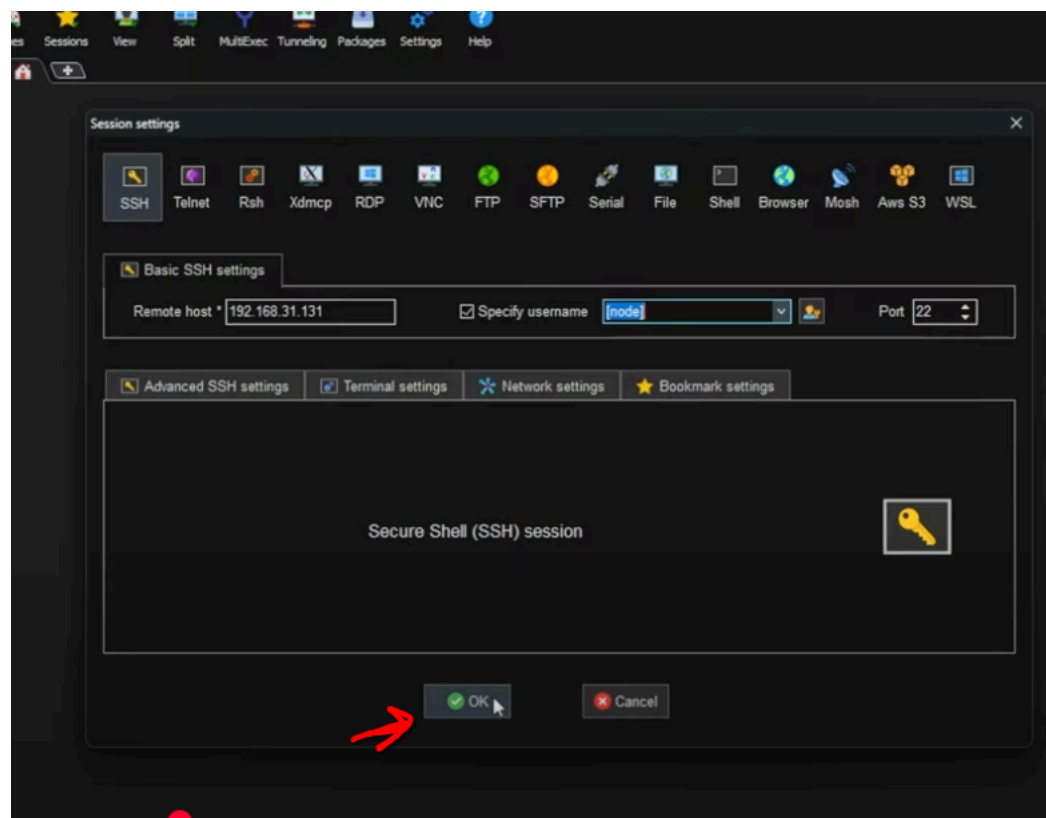
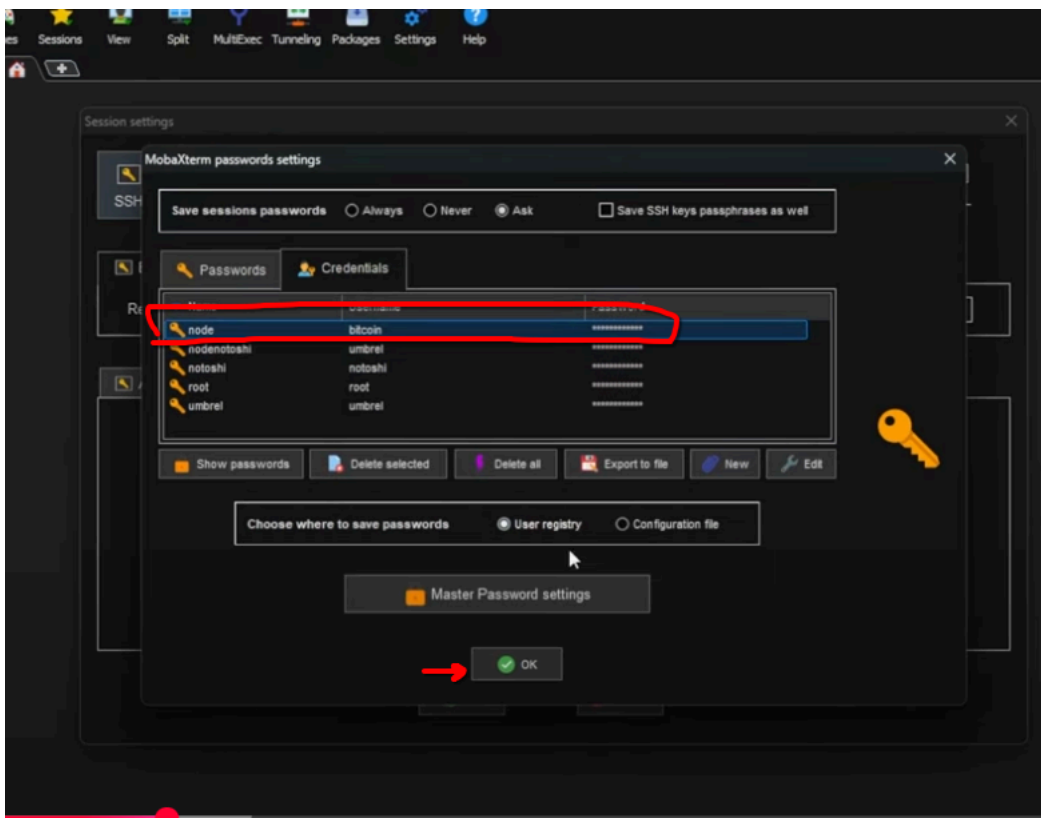
ip a

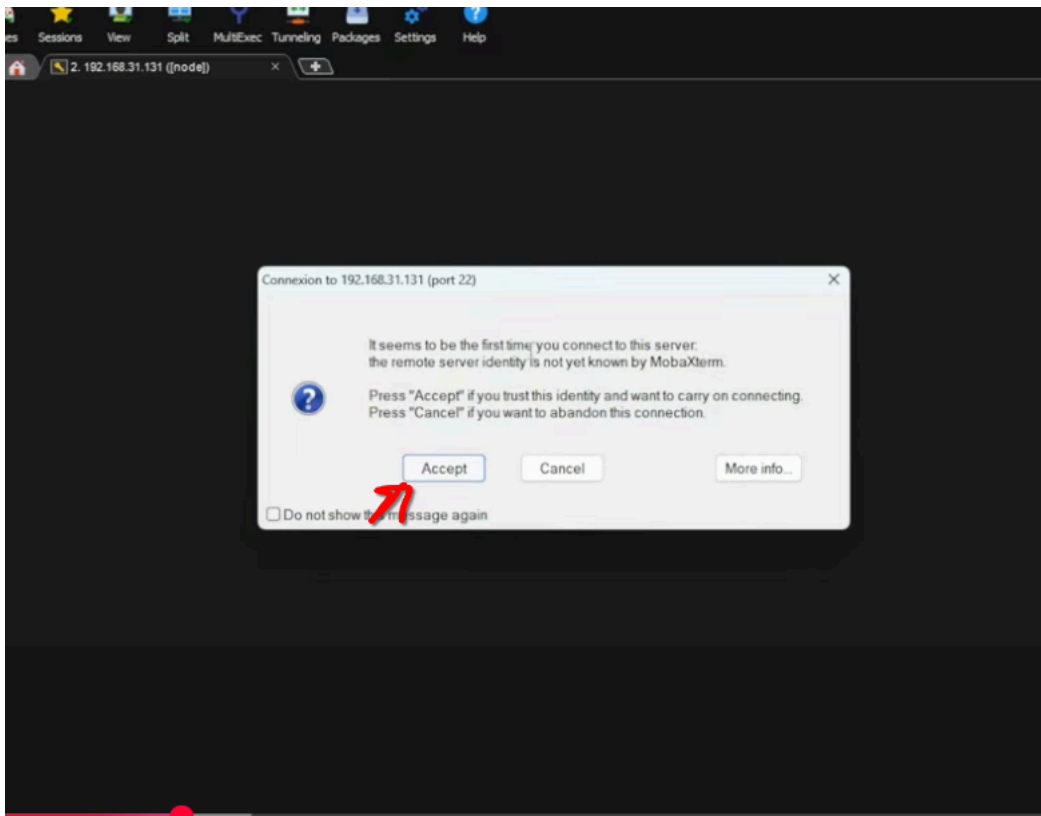
หรือดูได้จาก IP ที่ได้จดเอาไว้แล้วตอนลง Ubuntu server

3.2. กรอก Username และ Password ของโหนดที่ได้ตั้งเอาไว้ตอนติดตั้ง Ubuntu Server



ตั้งชื่อ connection ตามด้วย username และ password ของโหนดที่เราได้ตั้งเอาไว้แล้วตอนลง Ubuntu server





4.ลงโปรแกรมพื้นฐานของ Ubuntu Server

เป็นการเตรียม Node ของคุณให้พร้อมสำหรับการติดตั้ง Software ต่อจากนี้

4.1. Update และ Upgrade ระบบ

```
sudo apt update && sudo apt upgrade -y
```

4.2. ติดตั้งเครื่องมือที่จำเป็นต้องใช้งาน

```
sudo apt install wget curl gnupg tar ufw -y
```

4.3. ตั้งค่า Firewall ภายในโหนดของคุณ เพื่อความปลอดภัย

เปิด Port เท่าที่จำเป็นต้องใช้

```
sudo ufw allow 22/tcp comment 'ssh'
sudo ufw allow 9051/tcp comment 'tor'
sudo ufw allow 9050/tcp comment 'tor'
sudo ufw allow 8333/tcp comment 'Bitcoin core peer'
sudo ufw allow 8332/tcp comment 'Bitcoin core RPC'
```

เปิด Firewall

```
sudo ufw enable
```

ตรวจสอบ Port Firewall

```
sudo ufw status
```

5.ติดตั้ง Bitcoin core และ Tor

5.1. ติดตั้ง Bitcoin core

ดาวน์โหลด Bitcoin core ลงเครื่อง

```
wget https://bitcoincore.org/bin/bitcoin-core-29.0/bitcoin-29.0-x86_64-linux-gnu.tar.gz
```

ตรวจสอบว่า Bitcoin core ที่เราดาวน์โหลดมานั้นถูกต้องสมบูรณ์หรือไม่

ดาวน์โหลดลายเซ็นที่ใช้ตรวจสอบโปรแกรม

```
wget https://bitcoincore.org/bin/bitcoin-core-29.0/SHA256SUMS
```

```
wget https://bitcoincore.org/bin/bitcoin-core-29.0/SHA256SUMS.asc
```

```
curl -s "https://api.github.com/repositories/355107265/contents/builder-keys" | grep download_url | grep -oE "https://[a-zA-Z0-9./-]+" | while read url; do curl -s "$url" | gpg --import; done
```

ควรที่จะปรากฏข้อความ

```
gpg: key 17565732E08E5E41: 29 signatures not checked due to missing keys
gpg: /home/admin/.gnupg/trustdb.gpg: trustdb created
gpg: key 17565732E08E5E41: public key "Andrew Chow <andrew@achow101.com>" imported
gpg: Total number processed: 1
gpg:             imported: 1
gpg: no ultimately trusted keys found
[...]
```

จากนั้น ตรวจสอบความถูกต้องของลายเซ็น

```
gpg --verify SHA256SUMS.asc SHA256SUMS
```

ควรที่จะปรากฏข้อความ

```
gpg: Good signature from...
Primary key fingerprint:...
```

เมื่อลายเซ็นถูกต้อง ก็นำลายเซ็นมาตรวจสอบโปรแกรม

```
sha256sum --ignore-missing --check SHA256SUMS
```

ควรที่จะปรากฏข้อความ

```
bitcoin-29.0-x86_64-linux-gnu.tar.gz: OK
```

แตกไฟล์ Bitcoin core

```
tar -xzf bitcoin-29.0-x86_64-linux-gnu.tar.gz
```

ติดตั้ง Bitcoin core

```
sudo install -m 0755 -o root -g root -t /usr/local/bin bitcoin-29.0/bin/bitcoin-cli bitcoin-29.0/bin/bitcoind
```

ตรวจสอบเวอร์ชันของ Bitcoin core

```
bitcoind --version
```

ทดสอบ Bitcoin core

```
bitcoind -daemon
```

สั่ง Bitcoin core หยุดทำงาน

```
bitcoin-cli stop
```

ลบไฟล์ที่ไม่ได้ใช้งาน

```
sudo rm -r bitcoin-$VERSION bitcoin-29.0-x86_64-linux-gnu.tar.gz SHA256SUMS SHA256SUMS.asc
```

สร้างไฟล์ Bitcoin.conf

```
sudo nano .bitcoin/bitcoin.conf
```

แก้ไขไฟล์ Bitcoin.conf ตามนี้

```
# [core]
# Run in the background as a daemon and accept commands.
daemon=1
# Maintain a full transaction index, used by the getrawtransaction rpc call.
txindex=1

# [network]
# Use separate SOCKS5 proxy <ip:port|path> to reach peers via Tor hidden services. May be a local file path prefixed with 'unix:'
onion=127.0.0.1:9050
bind=127.0.0.1
```

```
onlynet=onion
listen=1

# [rpc]
# Accept command line and JSON-RPC commands.
server=1
rpcuser=username #เปลี่ยน username ของคุณเอง
rpcpassword=password #เปลี่ยน password ของคุณเอง
rpcbind=0.0.0.0:8332
rpcallowip=0.0.0.0/0
deprecatedrpc=accounts
```

****โดยแทนที่ username และ password ด้วย username และ password ของท่าน****

```
rpcuser=username #เปลี่ยน username ของคุณเอง
rpcpassword=password #เปลี่ยน password ของคุณเอง
```

กด Ctrl + S และ Ctrl + X เพื่อบันทึกและออกจากไฟล์

5.2. ติดตั้ง Tor

```
sudo apt install tor -y
```

แก้ไข Tor Configuration

```
sudo nano /etc/tor/torrc
```

แก้ไข Tor Configuration ตามนี้

```
ControlPort 9051
CookieAuthentication 1
CookieAuthFileGroupReadable 1
HiddenServiceDir /var/lib/tor/bitcoin/bitcoinrpc
HiddenServiceVersion 3
HiddenServicePort 8332 127.0.0.1:8332
```

กด Ctrl + S และ Ctrl + X เพื่อบันทึกและออกจากไฟล์

ย้าย Bitcoin core ของคุณไปทำงานภายใต้ Tor โดย

สร้างโฟลเดอร์

```
sudo mkdir -p /var/lib/tor/bitcoin/bitcoinrpc
```

เปลี่ยน Ownership และ Permission ของโฟลเดอร์

```
sudo chown -R debian-tor:debian-tor /var/lib/tor/bitcoin/bitcoinrpc
sudo chmod 700 /var/lib/tor/bitcoin/bitcoinrpc
```

เพิ่มผู้ใช้ โดยแทนที่ username ด้วย username ของโหนดคุณ ****โดยเปลี่ยนคำว่า username เป็น username ของโหนดท่าน****

```
sudo usermod -a -G debian-tor username
```

รีสตาร์ท Tor

```
sudo systemctl restart tor
```

5.3. ทำให้ Bitcoin core ทำงานตลอดเวลาด้วย Systemd Service

สร้าง Systemd service File

```
sudo nano /etc/systemd/system/bitcoind.service
```

แก้ไขไฟล์ ตามนี้ ****โดยเปลี่ยนคำว่า username เป็น username ของโหนดท่าน****

```
[Unit]
Description=Bitcoin daemon
After=network.target

[Service]
ExecStart=/usr/local/bin/bitcoind -daemon -conf=/home/username/.bitcoin/bitcoin.conf -datadir=/home/username/.bitcoin
ExecStop=/usr/local/bin/bitcoin-cli stop
User=username
Group=username
Type=forking
Restart=on-failure

[Install]
WantedBy=multi-user.target
```

กด Ctrl + S และ Ctrl + X เพื่อบันทึกและออกจากไฟล์

เปิดใช้งาน Bitcoin core

```
sudo systemctl enable bitcoind
sudo systemctl start bitcoind
```

ตรวจสอบว่า Bitcoin core ทำงานไหมหลังเปิดใช้งาน

```
sudo systemctl status bitcoind
```

หากทุกอย่างปกติ ควรที่จะปรากฏข้อความดังนี้

```
● bitcoind.service - Bitcoin daemon
   Loaded: loaded (/etc/systemd/system/bitcoind.service; enabled; preset: enabled)
   Active: active (running) since Sat 2025-04-19 14:10:16 UTC; 23min ago
     Process: 812 ExecStart=/usr/local/bin/bitcoind -daemon -conf=/home/node/.bitcoin/bitcoin.conf -datadir=/home/node/.bitcoin
(code=exited, >
   Main PID: 876 (bitcoind)
    Tasks: 26 (limit: 4552)
   Memory: 3.2G (peak: 3.2G swap: 252.0K swap peak: 252.0K)
      CPU: 12min 55.480s
   CGroup: /system.slice/bitcoind.service
           └─876 /usr/local/bin/bitcoind -daemon -conf=/home/node/.bitcoin/bitcoin.conf -datadir=/home/node/.bitcoin

Apr 19 14:10:14 node systemd[1]: Starting bitcoind.service - Bitcoin daemon...
Apr 19 14:10:16 node bitcoind[812]: Bitcoin Core starting
Apr 19 14:10:16 node systemd[1]: Started bitcoind.service - Bitcoin daemon.
```

5.4. คำสั่งในการตรวจสอบการทำงานต่าง ๆ ของ bitcoind (Bitcoin core)

ตรวจสอบว่าโหนดต่อกับอินเทอร์เน็ตอยู่หรือไม่

```
bitcoin-cli -netinfo
```

ควรที่จะปรากฏข้อความดังต่อไปนี้

```
      onion    total    block
in           0         0
out          10        10        2
total        10        10
```

```
Local addresses
address.onion    port    8333    score    4
```

ตรวจสอบ Port การเชื่อมต่อของโหนด

```
sudo ss -tulpn | grep bitcoind
```

ควรที่จะปรากฏข้อความดังต่อไปนี้

```
tcp    LISTEN 0      4096      127.0.0.1:8333      0.0.0.0:*    users:((("bitcoind",pid=876,fd=30))
tcp    LISTEN 0      128       0.0.0.0:8332      0.0.0.0:*    users:((("bitcoind",pid=876,fd=12))
```

ตรวจสอบว่าโหนดได้ดาวน์โหลดข้อมูลบนบล็อกเชนไปแล้วกี่เปอร์เซ็นต์

```
echo "$(bitcoin-cli getblockchaininfo | jq -r '.verificationprogress * 100') %"
```

6.เปิดใช้งาน rpc auth

การเปิดใช้งาน rpc auth นั้นทำเพื่อป้องกันไม่ให้ id และ password ของ Bitcoin core ของคุณหลุดออกไปภายนอก

6.1. ดาวน์โหลดโปรแกรมสำหรับสร้าง rpc auth

ไปที่เว็บไซต์ <https://github.com/bitcoin/bitcoin/blob/master/share/rpcauth/rpcauth.py>

กดปุ่ม Raw ดังภาพ

Code Blame Raw

```

1  #!/usr/bin/env python3
2  # Copyright (c) 2015-2021 The Bitcoin Core developers
3  # Distributed under the MIT software license, see the accompanying
4  # file COPYING or http://www.opensource.org/licenses/mit-license.php.
5
6  from argparse import ArgumentParser
7  from getpass import getpass
8  from secrets import token_hex, token_urlsafe
9  import hmac
10 import json
11
12 def generate_salt(size):
13     """Create size byte hex salt"""
14     return token_hex(size)
15
16 def generate_password():
17     """Create 32 byte b64 password"""
18     return token_urlsafe(32)
19
20 def password_to_hmac(salt, password):
21     m = hmac.new(salt.encode('utf-8'), password.encode('utf-8'), 'SHA256')
22     return m.hexdigest()
23
24 def main():
25     parser = ArgumentParser(description='Create login credentials for a JSON-RPC user')
26     parser.add_argument('username', help='the username for authentication')
27     parser.add_argument('password', help='leave empty to generate a random password or specify "-" to prompt for password')
28     parser.add_argument("-j", "--json", help="output to json instead of plain-text", action='store_true')
29     args = parser.parse_args()
30
31     if not args.password:
32         args.password = generate_password()
33     elif args.password == '-':
34         args.password = getpass()
35
36     # Create 16 byte hex salt
37     salt = generate_salt(16)
38     password_hmac = password_to_hmac(salt, args.password)
39
40     if args.json:
41         odict={'username':args.username, 'password':args.password, 'rpcauth':f'{args.username}:{salt}${password_hmac}'}
42         print(json.dumps(odict))
43     else:
44         print('String to be appended to bitcoin.conf:')
45         print(f'rpcauth={args.username}:{salt}${password_hmac}')
46         print(f'Your password:\n{args.password}')
47

```

เมื่อเข้าไปหน้าดังกล่าว ให้ Copy url ของหน้าเว็บนั้น

 raw.githubusercontent.com/bitcoin/bitcoin/refs/heads/master/share/rp...   

เข้าไปในหน้า Terminal

cd

พิมพ์คำสั่งดังกล่าว โดย past url ที่ได้ copy เอาไว้ลงไปต่อท้าย

wget https://raw.githubusercontent.com/bitcoin/bitcoin/refs/heads/master/share/rpcauth/rpcauth.py

หลังจากนั้นก็เช็คว่ามีโปรแกรมลงเครื่องเราหรือยัง ด้วย

ls

หากโปรแกรมอยู่ในเครื่องแล้ว ควรที่จะมีชื่อไฟล์ "rpcauth.py" ปรากฏขึ้น

Desktop Documents Downloads rpcauth.py

6.2. แก้ไข bitcoin.conf

เข้าไปในไฟล์ bitcoin.conf

nano ~/.bitcoin/bitcoin.conf

เพิ่มข้อความดังต่อไปนี้เข้าไปใน file

```
rpcuser=your_rpc_username  
rpcpassword=your_rpc_password
```

กด Ctrl + S และ Ctrl + X เพื่อบันทึกและออกจากไฟล์

Restart bitcoind

```
sudo systemctl restart bitcoind.service
```

ตรวจสอบสถานะการทำงานของ Bitcoin core

```
sudo systemctl status bitcoind.service
```

หาก Bitcoin core ทำงานปกติ ควรที่จะแสดงข้อความดังต่อไปนี้

```
● bitcoind.service - Bitcoin daemon  
   Loaded: loaded (/etc/systemd/system/bitcoind.service; enabled; preset: enabled)  
   Active: active (running) since Mon 2025-10-06 00:27:54 +07; 7s ago  
     Process: 52385 ExecStart=/usr/local/bin/bitcoind -daemon -conf=/home/pi/.bitcoin/bitcoin.conf -datadir=/home/pi/.bitcoin  
(code=exited, status=0/SUCCESS)  
    Main PID: 52387 (bitcoind)  
      Tasks: 25 (limit: 9572)  
       CPU: 8.349s  
    CGroup: /system.slice/bitcoind.service  
            └─52387 /usr/local/bin/bitcoind -daemon -conf=/home/pi/.bitcoin/bitcoin.conf -datadir=/home/pi/.bitcoin  
  
Oct 06 00:27:54 raspberrypi systemd[1]: Starting bitcoind.service - Bitcoin daemon...  
Oct 06 00:27:54 raspberrypi bitcoind[52385]: Bitcoin Core starting  
Oct 06 00:27:54 raspberrypi systemd[1]: Started bitcoind.service - Bitcoin daemon.
```

ตรวจสอบไฟล์ Log ของ bitcoin

```
$ tail -f ~/.bitcoin/debug.log
```

หากทำงานปกติ ควรที่จะแสดงข้อความดังต่อไปนี้

```
2025-10-05T17:28:02Z Progress loading mempool transactions from file: 80% (tried 765, 191 remaining)  
2025-10-05T17:28:02Z Progress loading mempool transactions from file: 90% (tried 861, 95 remaining)  
2025-10-05T17:28:02Z Imported mempool transactions from file: 956 succeeded, 0 failed, 0 expired, 0 already there, 0 waiting for  
initial broadcast  
2025-10-05T17:28:02Z initload thread exit  
2025-10-05T17:28:10Z New block-relay-only v1 peer connected: version: 70016, blocks=917790, peer=0  
2025-10-05T17:28:14Z New block-relay-only v2 peer connected: version: 70016, blocks=917790, peer=1  
2025-10-05T17:28:18Z New outbound-full-relay v1 peer connected: version: 70016, blocks=917790, peer=2  
...  
...
```

6.3. นำ rpc auth ที่ได้บรรจุลงไปใน bitcoin.conf

สั่งให้โปรแกรมสร้าง code rpc auth ทำงาน โดยแทนที่ <your_username> <your_password> ด้วย Username และ Password ของท่าน

```
python3 rpcauth.py <your_username> <your_password>
```

หลังจากนั้น ระบบก็จะขึ้นข้อความประมาณนี้ ให้ copy ข้อความหลัง rpcauth=<usernameของท่าน>:

```
String to be appended to bitcoin.conf:  
rpcauth=test:d4951c8a7434864791b284f1ab418eba$8ad6f178fdfa62f926cde43966ec1eadf6f3f5bb0afb5baed0b1dcba7f9d7  
Your password:
```

แล้วนำไปใส่ใน bitcoin.conf โดย แทนที่ test ด้วย username ของท่าน

```
nano ~/.bitcoin/bitcoin.conf  
  
rpcauth=test:59404178f93dda796d92c472090a5262$e12eb787cf0eef56bc8589342cfd428a361370403c031d51bf4c73fa6bb9bf3
```

กด Ctrl + S และ Ctrl + X เพื่อบันทึกและออกจากไฟล์

7. ติดตั้ง Electrum Server

Electrum Server นั้นมีไว้ให้สามารถเชื่อมต่อกับ Software wallet ส่วนใหญ่ที่นิยมใช้ และเพื่อความรวดเร็วในการใช้ Node

7.1. ลงตัวอ่านภาษา rust

ก่อนการติดตั้งตัวอ่าน rust เวอร์ชันเก่าเพื่อความแน่ใจว่าระบบจะไม่มีปัญหา


```
dpkg -l | grep rust
sudo apt remove --purge libstd-rust-1.63 libstd-rust-dev
sudo apt autoremove
```

ลงตัวตัวอ่านภาษา rust

```
curl --proto '=https' --tlsv1.2 -sSf https://sh.rustup.rs | sh
```

หลังจากนั้น ก็ตรวจสอบ version

```
which rustc
```

ควรจะปรากฏข้อความ : /home/pi/.cargo/bin/rustc

```
which cargo
```

ควรจะปรากฏข้อความ : /home/pi/.cargo/bin/cargo

```
rustc --version
```

ควรจะปรากฏข้อความ : rustc 1.89.0 (29483883e 2025-08-04)

```
cargo --version
```

ควรจะปรากฏข้อความ : cargo 1.89.0 (c24e10642 2025-06-23)

7.2. ติดตั้ง electrs (Electrum server)

ลง electrs เข้าโหนดของท่าน

```
sudo apt update
sudo apt install -y build-essential libclang-dev git # not install cargo
git clone https://github.com/romanz/electrs
cd electrs
cargo build --locked --release
```

หลังจากนั้น ก็ตรวจสอบ version

```
./target/release/electrs --version
```

หากโปรแกรม Electrs ลงเสร็จสิ้น ควรที่จะแสดงข้อความบอก version ของโปรแกรม

แก้ไข bitcoin.conf เพื่อให้ bitcoin core ของเรารองรับ electrum server

```
sudo ufw allow 50001/tcp comment 'allow electrs TCP from anywhere'
sudo ufw allow 50002/tcp comment 'allow electrs SSL from anywhere'
sudo ufw reload
```

ควรที่จะปรากฏข้อความ: Firewall reloaded

```
sudo ufw status
```

ควรที่จะปรากฏข้อความดังต่อไปนี้

To	Action	From	
--	-----	----	
...			
50002/tcp	ALLOW	Anywhere	# allow electrs SSL from anywhere
50001/tcp	ALLOW	Anywhere	# allow electrs TCP from anywhere
...			
50001/tcp (v6)	ALLOW	Anywhere (v6)	# allow electrs TCP from anywhere
50002/tcp (v6)	ALLOW	Anywhere (v6)	# allow electrs SSL from anywhere
...			

ต่อมา

```
nano ~/.bitcoin/bitcoin.conf
```

และเพิ่มข้อความดังต่อไปนี้เข้าไป

```
zmqpubrawblock=tcp://0.0.0.0:28332 # Enable publishing of raw block notifications
zmqpubrawtx=tcp://0.0.0.0:28333 # Enable publishing of raw transaction notifications
zmqpubhashblock=tcp://0.0.0.0:28334 # Enable publishing of block hash notifications
whitelist=127.0.0.1 # trust all connections from localhost" (your own machine)
```

กด Ctrl + S และ Ctrl + X เพื่อบันทึก

ทำสำเนาไฟล์ config_example.toml เข้าไปในโฟลเดอร์ของelectrum server

```
cp ~/electrs/doc/config_example.toml ~/electrs/electrs.toml
```

แก้ไขไฟล์ electrs.toml ที่เราได้ทำสำเนาเอาไว้

```
nano ~/electrs/electrs.toml
```

แก้ไขจาก

```
# File where bitcoind stores the cookie, usually file .cookie in its datadir
cookie_file = "/var/run/bitcoin-mainnet/cookie"

# Directory where the index should be stored. It should have at least 70GB of free space.
db_dir = "/some/fast/storage/with/big/size"

# The address on which electrs should listen. Warning: 0.0.0.0 is probably a bad idea!
# Tunneling is the recommended way to access electrs remotely.
electrum_rpc_addr = "127.0.0.1:50001"
```

เป็น

```
# File where bitcoind stores the cookie, usually file .cookie in its datadir
cookie_file = "/var/run/bitcoin-mainnet/cookie"

# Directory where the index should be stored. It should have at least 70GB of free space.
db_dir = "/some/fast/storage/with/big/size"

# The address on which electrs should listen. Warning: 0.0.0.0 is probably a bad idea!
# Tunneling is the recommended way to access electrs remotely.
electrum_rpc_addr = "127.0.0.1:50001"
```

ทดสอบว่า electrum server ทำงานปกติไหม

```
./target/release/electrs
```

หากทุกอย่างเรียบร้อย ควรที่จะปรากฏข้อความดังต่อไปนี้

```
Starting electrs 0.10.10 on aarch64 linux with Config { network: Bitcoin, db_path: "/home/pi/electrs/db/bitcoin", db_log_dir:
None, db_parallelism: 1, daemon_auth: CookieFile("/home/pi/.bitcoin/.cookie"), daemon_rpc_addr: 127.0.0.1:8332, daemon_p2p_addr:
127.0.0.1:8333,
electrum_rpc_addr: 0.0.0.0:50001, monitoring_addr: 127.0.0.1:4224, wait_duration: 10s, jsonrpc_timeout: 15s, index_batch_size: 10,
index_lookup_limit: None,
reindex_last_blocks: 0, auto_reindex: true, ignore_mempool: false, sync_once: false, skip_block_download_wait: false,
disable_electrum_rpc: false, server_banner: "Welcome to electrs 0.10.10 (Electrum Rust Server)!", signet_magic: f9beb4d9 }
[2025-10-06T15:06:08.347Z INFO electrs::metrics::metrics_impl] serving Prometheus metrics on 127.0.0.1:4224
[2025-10-06T15:06:08.347Z INFO electrs::server] serving Electrum RPC on 0.0.0.0:50001
[2025-10-06T15:06:08.387Z INFO electrs::db] "/home/pi/electrs/db/bitcoin": 2 SST files, 0.000002204 GB, 0.000000002 Grows
[2025-10-06T15:06:08.396Z INFO electrs::index] indexing 2000 blocks: [1..2000]
[2025-10-06T15:06:08.521Z INFO electrs::chain] chain updated: tip=00000000df5d65c9d8561b4b8f60a63018fe3933ecb131fb37f905f87da951a,
height=2000
[2025-10-06T15:06:08.525Z INFO electrs::index] indexing 2000 blocks: [2001..4000]
```

จากนั้นทำให้ electrs ทำงานตลอดเวลา โดย

```
sudo nano /etc/systemd/system/electrs.service
```

แก้ไขไฟล์ดังนี้

```
[Unit]
Description=Electrs
After=bitcoind.service # Ensures electrs starts only after bitcoind is running

[Service]
WorkingDirectory=/home/bitcoin/electrs # Path to electrs directory change bitcoin to your user
ExecStart=/home/bitcoin/electrs/target/release/electrs # path to execute electrs change bitcoin to your user
User=bitcoin # change bitcoin to your user
Group=bitcoin # change bitcoin to your user
Type=simple
KillMode=process
TimeoutSec=60
Restart=always
RestartSec=60

Environment="RUST_BACKTRACE=1"

# Hardening measures
PrivateTmp=true
ProtectSystem=full
NoNewPrivileges=true
MemoryDenyWriteExecute=true

[Install]
WantedBy=multi-user.target
```

และกด Ctrl + S และ Ctrl + X เพื่อบันทึกและออก

จากนั้นก็ทำการเปิดใช้งาน electrs.service เพื่อให้ electrum server ทำงานตลอดเวลา

```
sudo systemctl daemon-reload
sudo systemctl enable electrs.Service
sudo systemctl start electrs.service
sudo systemctl status electrs.service
```

7.3. ย้าย Electrum server ให้ทำงานผ่าน Tor

เข้าแก้ไข file

```
sudo nano /etc/tor/torrc
```

เพิ่มข้อความดังต่อไปนี้เข้าไปในไฟล์

```
# Electrs
HiddenServiceDir /var/lib/tor/electrs
HiddenServiceVersion 3
HiddenServicePort 50001 127.0.0.1:50001
HiddenServiceEnableIntroDoSDefense 1
```

กด Ctrl + S และ Ctrl + X เพื่อบันทึกและออกจากไฟล์

จากนั้น

```
sudo mkdir -p /var/lib/tor/electrs
sudo chown -R debian-tor:debian-tor /var/lib/tor/electrs
sudo chmod -R 700 /var/lib/tor/electrs
sudo systemctl restart tor
sudo systemctl status tor
```

และตรวจสอบ Tor address ของเรา

```
sudo cat /var/lib/tor/electrs/hostname
```

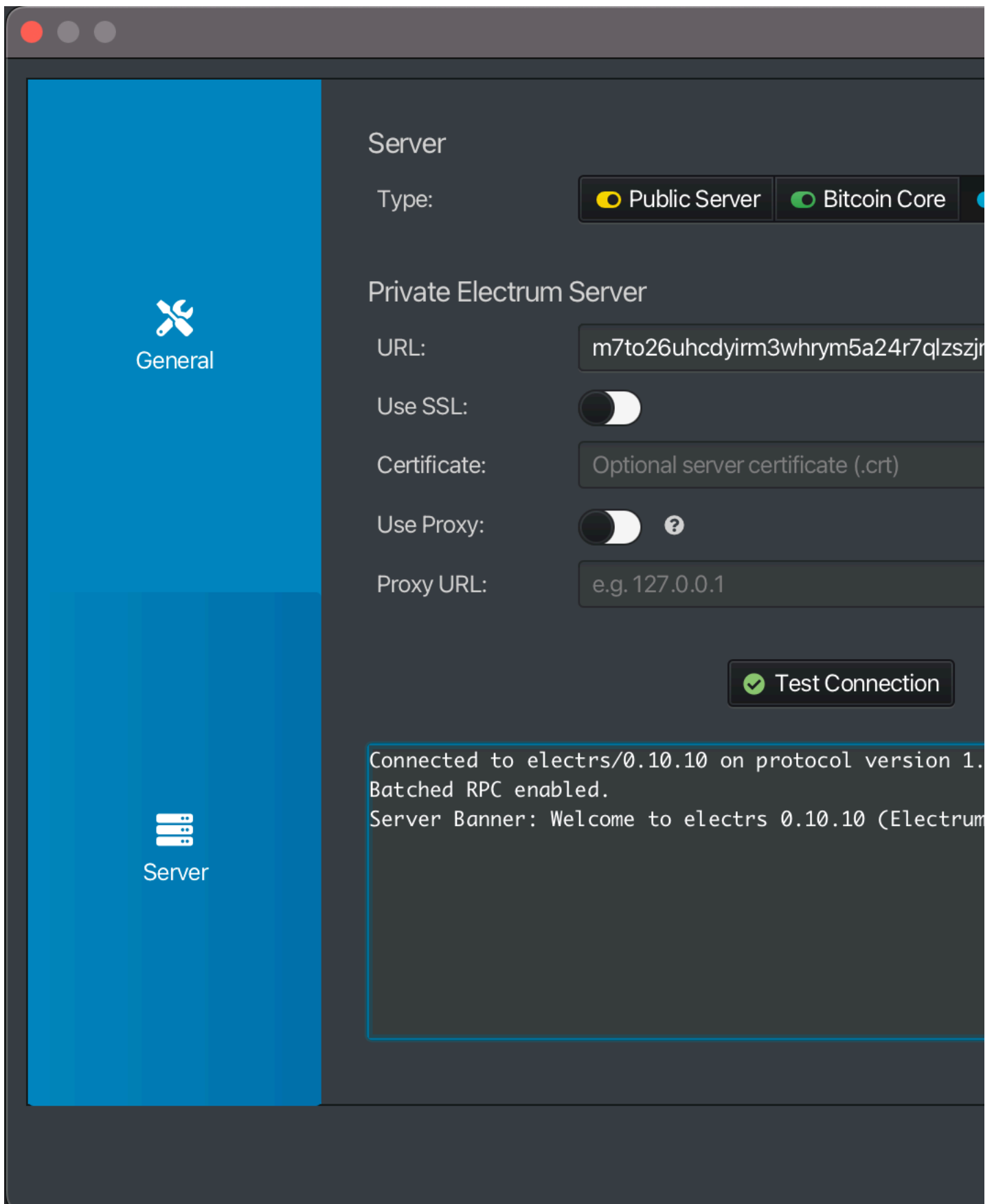
ควรที่จะปรากฏข้อความ .onion

```
m7to26uhcdyirm3whrym5a24r7qlzsjrzqbfurx2gemvwgcs6xt7gqd.onion
```

8.เชื่อมต่อโหนดของคุณเข้ากับ Sparrow

พิมพ์คำสั่งนี้ใน Terminal เพื่อดู Tor Address ของท่าน แล้วนำไปกรอกตามภาพแล้วคลิกที่ Test Connection

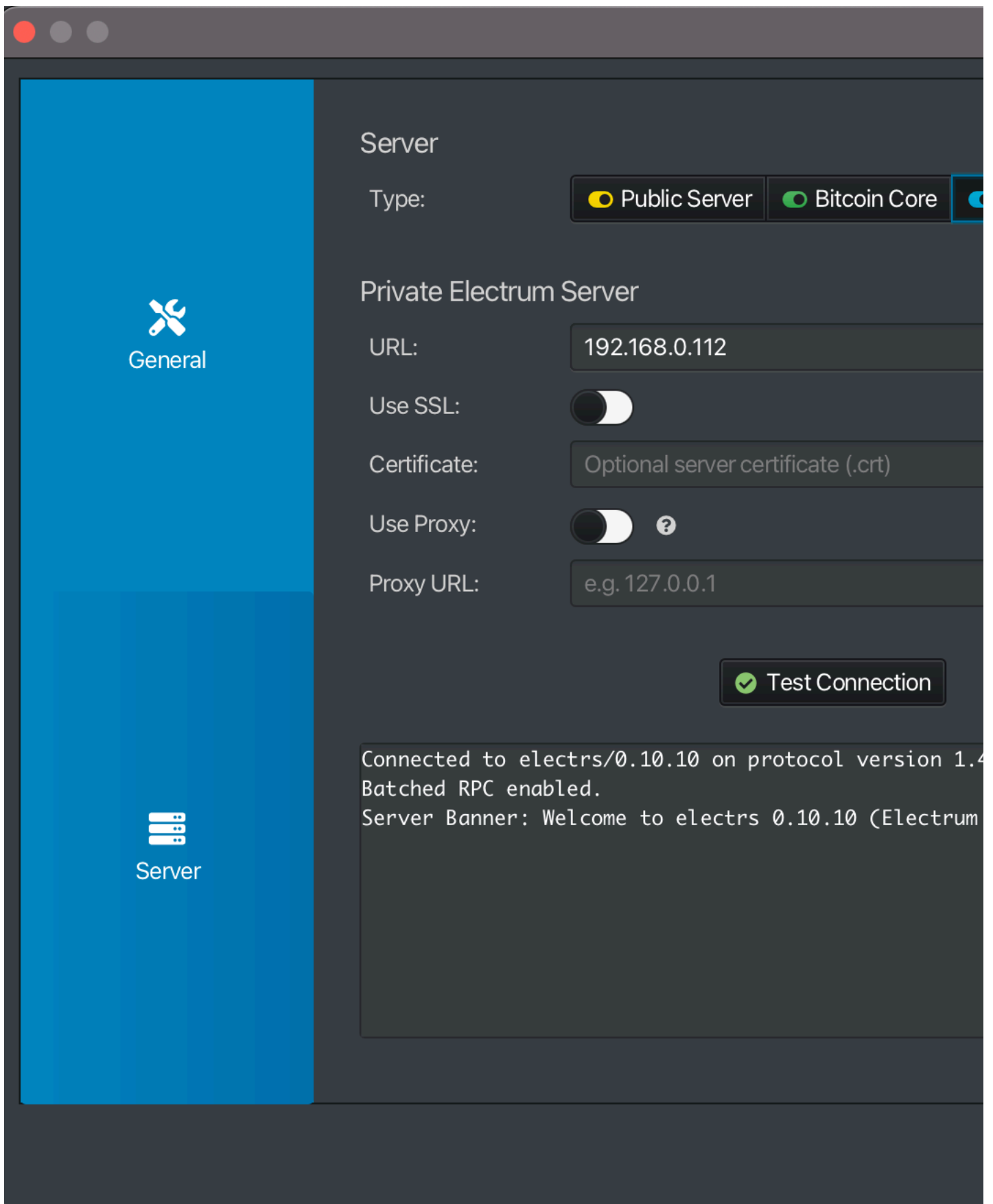
```
sudo cat /var/lib/tor/electrs/hostname
m7to26uhcdyirm3whrym5a24r7qlzsjrzqbfurx2gemvwgcs6xt7gqd.onion
```



หรือใช้คำสั่งนี้เพื่อดู ip แล้วเชื่อมต่อผ่าน ip แล้วนำไปกรอกตามภาพแล้วคลิกที่ Test Connection

```
ip a | grep wlan0
3: wlan0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc noqueue state UP group default qlen 1000
    inet 192.168.0.112/24 brd 192.168.1.255 scope global dynamic noprefixroute wlan0
```

จากตัวอย่าง ip ที่ได้คือ 192.168.0.112



9.ติดตั้ง Bitcoin RPC Explorer

Bitcoin RPC Explorer นั้นเป็นโปรแกรมที่ใช้ดูข้อมูลใน Blockchain ของ Bitcoin ได้เหมือน mempool.space เช่น ดูข้อมูลธุรกรรมของท่าน หรือดูระดับความยากในการปิดบล็อกปัจจุบัน

9.1. ติดตั้ง Node.js

```
# Download and install nvm:
curl -o- https://raw.githubusercontent.com/nvm-sh/nvm/v0.40.3/install.sh | bash

# in lieu of restarting the shell
\ . "$HOME/.nvm/nvm.sh"

# Download and install Node.js:
nvm install 22

# Verify the Node.js version:
node -v # Should print "v22.20.0".

# Verify npm version:
npm -v # Should print "10.9.3".
```

9.2. ติดตั้ง Bitcoin RPC Explorer

ดึงโปรแกรมจาก Github เข้ามาในโหนดของเรา

```
cd

git clone https://github.com/janoside/btc-rpc-explorer.git
cd btc-rpc-explorer
```

```
ลง npm

npm install
```

แก้ไขไฟล์ .env ของ Bitcoin RPC Explorer

```
cp .env-sample .env
nano .env
```

แก้ไขโดยลบ # ที่บรรทัดตามที่แสดงดังต่อไปนี้

```
BTCEXP_BITCOIND_HOST=<ip-of-your-bitcoin-node>
BTCEXP_BITCOIND_PORT=<rpc-port-of-your-bitcoin-node>
BTCEXP_BITCOIND_USER=<your-rpc-username>
BTCEXP_BITCOIND_PASS=<your-rpc-password>
BTCEXP_BITCOIND_COOKIE=/path/to/bitcoind/.cookie
```

และบรรทัดดังต่อไปนี้เพื่อให้ Bitcoin RPC Explorer ทำงานกับ Electrum Server ได้

```
BTCEXP_ADDRESS_API=electrum
BTCEXP_ELECTRUM_SERVERS=tcp://0.0.0.0:50001
BTCEXP_ELECTRUM_TXINDEX=true
BTCEXP_RATE_LIMIT_WINDOW_MINUTES=-1
BTCEXP_RATE_LIMIT_WINDOW_MAX_REQUESTS=0
```

แล้วจึงกด Ctrl + S และ Ctrl + X เพื่อบันทึกและออกจากไฟล์

ปรับเปลี่ยน Firewall ให้โหนดเราใช้ Bitcoin RPC Explorer ได้

```
sudo ufw allow 3002/tcp comment 'btc-rpc-explorer'
sudo ufw reload
```

ควรที่จะปรากฏข้อความ: Firewall reloaded

```
sudo ufw status
```

ควรที่จะปรากฏข้อความดังต่อไปนี้

To	Action	From	
--	-----	----	
...			
50002/tcp	ALLOW	Anywhere	# allow electrs SSL from anywhere
50001/tcp	ALLOW	Anywhere	# allow electrs TCP from anywhere
...			
50001/tcp (v6)	ALLOW	Anywhere (v6)	# allow electrs TCP from anywhere
50002/tcp (v6)	ALLOW	Anywhere (v6)	# allow electrs SSL from anywhere
...			

สุดท้าย เปิดใช้งาน Bitcoin RPC Explorer

```
npm start
```

จากนั้น ทำให้ Bitcoin RPC Explorer ทำงานตลอดเวลาโดย

```
sudo vim /etc/systemd/system/btcrpcexplorer.service
```

เพิ่มข้อความดังต่อไปนี้เข้าไปในไฟล์ **โดยแทนที่ คำว่า pi ด้วย username ของท่าน และ /home/pi/btc-rpc-explorer ด้วยที่อยู่ btc-rpc-explorer ภายในโหนดของท่าน**

```
[Unit]
Description=BTC RPC Explorer
Requires=bitcoind.service electrs.service
After=bitcoind.service electrs.service

[Service]
User=pi
Group=pi
WorkingDirectory=/home/pi/btc-rpc-explorer

Environment="NVM_DIR=/home/pi/.nvm"
Environment="PATH=/home/pi/.nvm/versions/node/v22.20.0/bin:/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin"

Environment=NODE_ENV=production
ExecStart=/home/pi/.nvm/versions/node/v22.20.0/bin/npm start

Restart=on-failure
RestartSec=3

PrivateTmp=true
ProtectSystem=full
NoNewPrivileges=true
PrivateDevices=true

[Install]
WantedBy=multi-user.target
```

กด Ctrl + S และ Ctrl + X เพื่อบันทึกและออกจากไฟล์

ต่อมา

```
sudo systemctl daemon-reload
sudo systemctl enable btc-rpc-explorer
sudo systemctl start btc-rpc-explorer
sudo systemctl status btc-rpc-explorer
```

สุดท้าย

```
sudo systemctl status btc-rpc-explorer.service
```

หากโปรแกรมทำงานปกติ ควรจะปรากฏข้อความดังต่อไปนี้

```
● btc-rpc-explorer.service - BTC RPC Explorer
   Loaded: loaded (/etc/systemd/system/btc-rpc-explorer.service; enabled; preset: enabled)
   Active: active (running) since Fri 2025-10-10 10:10:47 +07; 1min 8s ago
     Main PID: 152082 (npm start)
        Tasks: 23 (limit: 9572)
           CPU: 1.719s
    CGroup: /system.slice/btc-rpc-explorer.service
            └─152082 "npm start"
                └─152093 sh -c "node ./bin/www"
                    └─152094 node ./bin/www
```
